

The ledger

An arena sweep for S -adic density on $y^2 = f(x)f(t)$: what the algorithm computes, how the code implements it, and why it need not terminate

companion to ledger.gp

1. What the ledger is for

Fix a finite set S of primes. We want to decide whether $X(\mathbb{Q})$ is dense in $\prod_{p \in S} X(\mathbb{Q}_p)$ — S -adic density, the several-places-at-once question. Two features of X shape the algorithm.

Points of X come in pairs, from a single twist. Since $X(\mathbb{Q}) = \sqcup_d (E_d \times E_d)(\mathbb{Q})/\pm$, a rational point of X is an **ordered pair** of points on **one** twist E_d . So the covering condition is not about covering a set of local points but about covering **pairs**: for every pair of local targets there must be a single twist carrying rational points near both.

No single twist need suffice. Different d contribute different subgroups, and they can patch together: a family of twists none of which is dense on its own can still cover every pair between them. That is the phenomenon the ledger is built to detect, and it is why a twist-by-twist search — which is what Section 1’s alternative would be — misses it.

The plan. Replace each local group by a finite quotient (the **arena**), replace each twist by the subgroup its rational points generate there (its **reach**), and ask whether the reaches cover all ordered pairs. A finite list of reaches that does is a **certificate**: a finite, checkable datum implying density.

2. The arena

The arena is $A = \prod_{p \in S} G_p$, where G_p is a finite quotient of $E_\delta(\mathbb{Q}_p)$. At level 1 with good reduction $G_p = \tilde{E}(\mathbb{F}_p)$ and everything is reduction of points; in general `ledger.gp` uses a **reduction-agnostic** arena built from explicit coset representatives, so that bad reduction and the deeper layers are handled uniformly.

`arenainit` builds, for each place, the list of coset representatives together with **precomputed addition and negation tables**, so that all later arithmetic is table lookup on small integers:

```
arenainit(A, B, d0, S) = {
  my(dat = List(), ...);
  Es = shortmodel(A, B, d0);
  for(i = 1, #S,
    cr = cosetreps1(Es, S[i], 3, PRECL);
    Ep = cr[1]; R = cr[2]; m = #R;
    add = matrix(m, m);
    for(j = 1, m, for(k = j, m,
      Q = if(R[j] == [0], R[k], if(R[k] == [0], R[j], elladd(Ep, R[j], R[k])));
      add[j,k] = cosetidx(Ep, R, S[i], Q); add[k,j] = add[j,k]);
    neg = vector(m, j, if(R[j] == [0], 1, cosetidx(Ep, R, S[i], ellneg(Ep, R[j]))));
    listput(dat, [Ep, R, m, add, neg, S[i]]);
  Vec(dat);
}
```

An arena element is a **mixed-radix integer**: `pack` and `unpack` convert between the tuple of per-place indices and a single integer in $[0, N)$, $N = \prod_p \#G_p$. Group operations are then `arenaadd` and `arenaneg`, coordinatewise table lookups.

3. The reach of a twist

Given the known rational points of E_d , the **reach** $R(d)$ is the subgroup of A they generate. The twist enters through $\lambda = \sqrt{d_0/d}$, which carries a point of E_d to a point of the fixed model E_{d_0} over $\mathbb{Q}_p$; **reachmap** then closes the generated set under addition and returns a bitmap of length N :

```
reachmap(ar, d, d0, S, pts) = {
  my(N = arenasize(ar), bm = vectorsmall(N), gens = List(), lam = lambdas(ar, d, d0));
  for(j = 1, #pts,
    listput(gens, pack(ar, vector(#S, i, redpoint(ar, i, lam[i], pts[j])))));
  bm[1] = 1; /* identity has index 0 */
  new = 1;
  while(new, new = 0;
    for(k = 0, N-1,
      if(bm[k+1] == 0, next);
      for(j = 1, #gens,
        cur = arenaadd(ar, k, gens[j]);
        if(bm[cur+1] == 0, bm[cur+1] = 1; new = 1)));
  bm;
}
```

A bitmap is the right representation: reaches are compared by containment and intersected constantly, and both are bitwise.

4. The sign action, and pruning

The Kummer involution acts, and so does the choice of square root at each place: negating the chosen λ_p replaces the reach by its image under negation in the p -th coordinate. So each computed reach yields $2^{\#S}$ reaches, one per sign vector, all equally valid. **signact** applies one; **ledgeradd** adds all of them and keeps only the **maximal** ones:

```
/* A ledger entry is [bitmap, d, eps]: the reach together with the twist and
   the sign that produced it. The provenance is what turns a closed ledger
   into a certificate -- a finite list of explicit twists whose reaches
   already cover the arena. */
ledgeradd(L, ar, bm, d) = {
  for(e = 0, 2^S - 1,
    eps = vector(S, i, if(bitand(e, 2^(i-1)), -1, 1));
    listput(cand, [signact(ar, bm, eps), d, eps]));
  for(j = 1, #cand,
    b = cand[j][1]; keep = 1;
    for(i = 1, #L, if(bmcontains(L[i][1], b), keep = 0; break()));
    if(keep, listput(L, cand[j])));
  /* prune members now dominated */
  ...
}
```

Pruning is what keeps the ledger small: it never grows beyond the number of **maximal** reaches seen, which in practice is a handful even after hundreds of twists.

5. The star test

Coverage is a condition on ordered pairs, and the implementation turns it into a condition on **membership masks**. For each arena element k , **maskvec** records the set of ledger indices whose reach contains k , as a bitmask. Then a pair (k, l) is covered exactly when **mask**[k] and **mask**[l] share a bit — some single reach contains both. So:

Coverage holds if and only if the distinct masks pairwise intersect.

```

starttest(L, N) = {
  my(mk = maskvec(L, N), D = Map(), ...);
  for(k = 1, N, c = if(mapisdefined(D, mk[k]), mapget(D, mk[k]), 0); mapput(D, mk[k], c+1));
  ks = Mat(D)[,1];
  for(i = 1, #ks, for(j = 1, #ks,
    tot += mapget(D, ks[i]) * mapget(D, ks[j]);
    if(bitand(ks[i], ks[j]) == 0,
      bad += mapget(D, ks[i]) * mapget(D, ks[j]))));
  [bad == 0, #ks, bad];
}

```

Two things make this cheap. Collapsing arena elements to **distinct masks** replaces a loop over N^2 pairs by a loop over the (usually far smaller) number of distinct membership patterns, weighted by multiplicity. And past a threshold the code abandons the exact count and returns only the boolean, since the quadratic loop over many distinct masks is the one place the algorithm can blow up.

The **deficiency** is **bad**: the number of ordered pairs no single reach covers. It is the quantity to watch, and Section 8 is about its behaviour.

6. The driver

run tally sweeps squarefree d in the square-class tuple of d_0 , computes each reach, adds it, and reports:

```

for(n = 1, DMAX,
  if(!issquarefree(n), next);
  for(sg = 0, 1,
    d = if(sg == 0, n, -n);
    if(sqclassS(d, S) != k0, next);
    td = twistdata(A, B, d);
    if(#td[2] == 0, next);
    bm = reachmap(ar, d, d0, S, td[2]);
    if(bmsize(bm) <= 1, next);
    L = ledgeradd(L, ar, bm, d); cnt++;
  ...))

```

The worked case of the README is $f = x^3 + x + 1$, $S = \{11, 13, 17\}$, arena $14 \times 18 \times 18 = 4536$: no twist is full, yet the ledger closes at seven maximal reaches, each of index 2 — the number of index-2 subgroups of $(\mathbb{Z}/2)^3$. Deficiency falls $74.8\% \rightarrow 28.1\% \rightarrow 0$ after 115 twists.

7. Grading, and what is actually proved

A tally proves nothing. Storing a reach by its image at one fixed level n — a **tally** entry, as opposed to a ledger entry — records an **over**-approximation: the preimage of the image can be strictly larger than the reach. Coverage computed from over-approximations is a necessary condition only, and level $n + 1$ can destroy what level n certified. No amount of computing at a fixed level, or at every level in turn, is a proof.

The fix is to make the level part of the datum, which is what turns a tally into a **ledger**. A ledger entry is $(d, n_d, \overline{R})$ subject to

$$R(d) \supseteq \ker_{n_d},$$

a **certificate of exactness**: the reach is then exactly the preimage of its image, and the finite datum is exact rather than approximate. If a finite graded ledger covers at $N = \max_d n_d$, the covering pulls back on the nose and density follows.

Granularity is diagonal. Write the arena as a grid with the places $p \in S$ as columns and the primes $\ell \mid \#G$ as rows. Off-diagonal cells are finite and $\ker_n$ is pro- p at each place, so it lives entirely on the diagonal:

granularity is a vector (n_p) , one per place, with $n_p = 1$ exactly when the reach's p -layer is everything. The test for $n_p = 1$ is direct:

```
gran1(Em, pts, S) = { my(i); for(i = 1, #S, if(!hitsEl(Em, pts, S[i]), return(0))); 1; }
```

and `rungraded` admits only granularity-1 twists. On the worked case the ledger still closes — 102 of 119 twists qualify — so the restriction costs little in practice.

Note the asymmetry. A closed ledger is a proof of density. A ledger that has not closed after $|d| \leq D$ is not evidence against it: it means no certificate has been found yet. The method certifies and never refutes.

8. Termination

Two questions, both raised by Rene, and the answers are: no, it need not terminate; yes, that forces unbounded granularity; and no, we cannot rule it out.

8.1. At a fixed level it does stabilise

If the level is **fixed** at N , the arena A_N is a finite group, the deficiency is an integer in $[0, \#A_N^2]$, and adding entries never increases it. A non-increasing sequence of non-negative integers is eventually constant. So at a fixed level the deficiency stabilises after finitely many twists — either at 0, when the ledger closes, or at a positive value it will never leave.

The same argument bounds the ledger itself: there are only finitely many subgroups of A_N , so only finitely many possible maximal reaches.

8.2. The graded ledger need not stabilise

The graded ledger is different, because the level is part of the datum and is **not** bounded in advance. Admitting an entry of granularity $n_d > N$ refines the arena, and the deficiency must be recomputed in the larger $A_{\max n_d}$. The right setting is then the profinite arena $A_\infty = \prod_{p \in S} E_\delta(\mathbb{Q}_p)$ with its Haar measure, where the covered set is

$$C_k = \bigcup_{i \leq k} R_i \times R_i \subseteq A_\infty \times A_\infty$$

and the deficiency is $1 - \mu(C_k)$. By countable additivity $\mu(C_k) \rightarrow \mu(\bigcup_i R_i \times R_i)$, and **nothing forces that limit to be attained**. So:

The deficiency can decrease through infinitely many distinct values, tending to 0 without ever reaching it. The ledger then never closes, even though density holds.

8.3. Non-termination forces unbounded granularity

This is Rene's second point, and it is right, with a one-line proof.

If the granularities of the admitted entries are bounded, say $n_d \leq N_0$ for all d , then the ledger stabilises after finitely many twists.

Proof. Each entry satisfies $R(d) \supseteq \ker_{N_0}$, so every reach is a union of cosets of $\ker_{N_0}$ and is determined by its image in the **finite** group A_{N_0} . There are finitely many subgroups of A_{N_0} , hence finitely many possible reaches and finitely many possible ledgers; and the deficiency, computed in A_{N_0} , is a non-increasing integer. ■

Contrapositive: a ledger that never stabilises must admit entries of unbounded granularity — $\max_p n_p \rightarrow \infty$ along the sweep.

8.4. Why it cannot be ruled out

Three remarks, in increasing order of how much they close the door.

Only finite-index reaches can ever appear. The certificate condition $R(d) \supseteq \ker_{n_d}$ is the statement that $R(d)$ is open, i.e. of finite index. A reach of infinite index — the typical situation when E_d has rank below the number g of topological generators of $\prod_p E_\delta(\mathbb{Q}_p)$ — admits no certificate at any level. Those twists are invisible to the graded ledger, however many rational points they carry. This is not a defect of the implementation: a finite family of closed subgroups covering a compact group must contain an open one, by Baire, so a closing ledger **has** to be made of finite-index reaches.

Unbounded granularity means vanishing contributions. If $n_p \rightarrow \infty$ then $[A_\infty : R_i] \rightarrow \infty$ and $\mu(R_i) \rightarrow 0$, so the pair-measure $\mu(R_i \times R_i)$ contributed by the tail tends to 0 fast. A never-closing sweep is therefore one where the bounded-granularity entries do almost all the work and an infinite tail supplies the last ε — slow convergence, and no finite stage attaining it.

And whether that happens is not something we control. Which subgroups occur as reaches is determined by the Mordell–Weil groups of the twists and by the p -adic positions of their generators. Whether the twists of granularity $\leq N_0$ suffice to cover, for some N_0 , is a question about rank distributions in a quadratic twist family and about how deep in the local filtration the generators sit. Even the existence of infinitely many twists of rank $\geq g$ is open in general. So there is no argument available — and, we think, none in sight — that bounds the granularity needed.

Summary. The ledger is a **semi-decision procedure** for S -adic density. When it closes it produces a finite certificate and a proof. When it does not, the sweep is simply unfinished, and the possibility Rene raises — deficiency shrinking forever without reaching 0, granularity unbounded — is a legitimate one that we can neither exhibit nor exclude.

9. A bug this documentation turned up

Writing Section 4 exposed a real defect. `ledgeradd` takes the twist d as its fourth argument precisely so that a closed ledger carries its provenance — the list of explicit twists whose reaches cover. But `runtally` called it with three arguments, and PARI silently supplies 0 for the missing one, so every entry recorded $d = 0$ and `certtwists` returned `[0]`: the certificate named no twists at all. The graded drivers `rungraded` and `sweepgraded` passed d correctly, so only the flat driver was affected — and only its provenance, never its arithmetic, since d is not used in the covering computation. Fixed; `certtwists` on the worked case now returns `[-274, -127, -87, 53]`.